Documento de Arquitetura de Solução (DAS)

1. Introdução

1.1 Propósito

Este documento descreve a arquitetura da solução de integração entre a Nexus e a Pay Cobranças, detalhando componentes, interações e decisões técnicas que suportam o processamento de faturas via Webhooks e filas assíncronas em Azure.

1.2 Escopo

O escopo contempla somente a definição arquitetural da solução (modelo C4, requisitos, componentes, riscos e métricas), não abrangendo implementação, testes ou operação em produção

1.3 Público-alvo

Destina-se a arquitetos de software, desenvolvedores, analistas de negócios, equipe de infraestrutura e stakeholders técnicos e de negócio envolvidos na evolução da integração Nexus–Pay Cobranças.

2. Visão Geral da Solução

2.1 Descrição Geral

A solução proposta é a implementação de uma application programming interface(API) que será responsável por receber notificações da Pay cobranças(Webhook). O objetivo principal é reduzir o tempo de processamento das faturas criadas por meio da integração e com isso passar dados financeiros fiéis de recebimentos e fluxo de caixa.

2.2 Requisitos Funcionais

- Receber notificação de pagamento/cancelamento via Webhook

- Registrar em fila os eventos recebidos

- Atualizar status das faturas no domínio financeiro da Nexus

- Permitir reprocessar eventos de pagamento/cancelamento em caso de falha

- Disponibilizar logs detalhados das transações de integração

- Enviar alertas à equipe quando houver falhas de integração acima de limiar

- Permitir parametrizar janelas de reenvio/reprocessamento em caso de inconsistência

- Integrar-se ao pipeline de CI/CD para implantação automatizada dos componentes de integração.

2.3 Requisitos Não Funcionais

Disponibilidade

- A solução de Webhook deve garantir disponibilidade mínima de 99,5% em curto prazo

- A integração de ponta a ponta(Nexus-Pay cobranças_ deve evoluir para a SLA de 99,9% em longo prazo.

Desempenho e escalabilidade

- O sistema deve reduzir em pelo menos 80% o tempo médio de processamento de faturas pagas e canceladas em relação ao cenário atual.

- As consultas SQL devem ser otimizadas para reduzir em cerca de 70% as leituras desnecessárias no banco de dados.

- O uso de filas e processamento assíncrono deve reduzir em aproximadamente 60% o tráfego de rede entre Nexus e Pay cobranças.

Confiabilidade e monitoramento

- O sistema deve garantir capacidade de reprocessamento de mensagens em caso de falhas, evitando perda de eventos de pagamento/cancelamento.

- Deve ser configuradas métricas e alertas em tempo real para indisponibilidade, aumento de erros, filas acumuladas e degradação de performance.

- A solução deve possuir mecanismos de failover automático para o serviço de Webhook e componentes críticos.

Segurança e conformidade

- Dados sensíveis de transações financeiras devem ser protegidos em trânsito(HTTPS/TLS).

- A arquitetura deve prever trilhas de auditoria(quem alterou o quê e quando) e atender requisitos de conformidade aplicáveis ao tratamento de dados financeiros e pessoais.

Arquitetura e tecnologia

- A solução deve evitar dependências de tecnologias não suportadas pela infraestrutura atual, mantendo aderência a padrões corporativos.

- A solução deve ser cloud-native e implementada em infraestrutura Azure, utilizando serviços gerenciados compatíveis com o ecossistema atual da Nexus

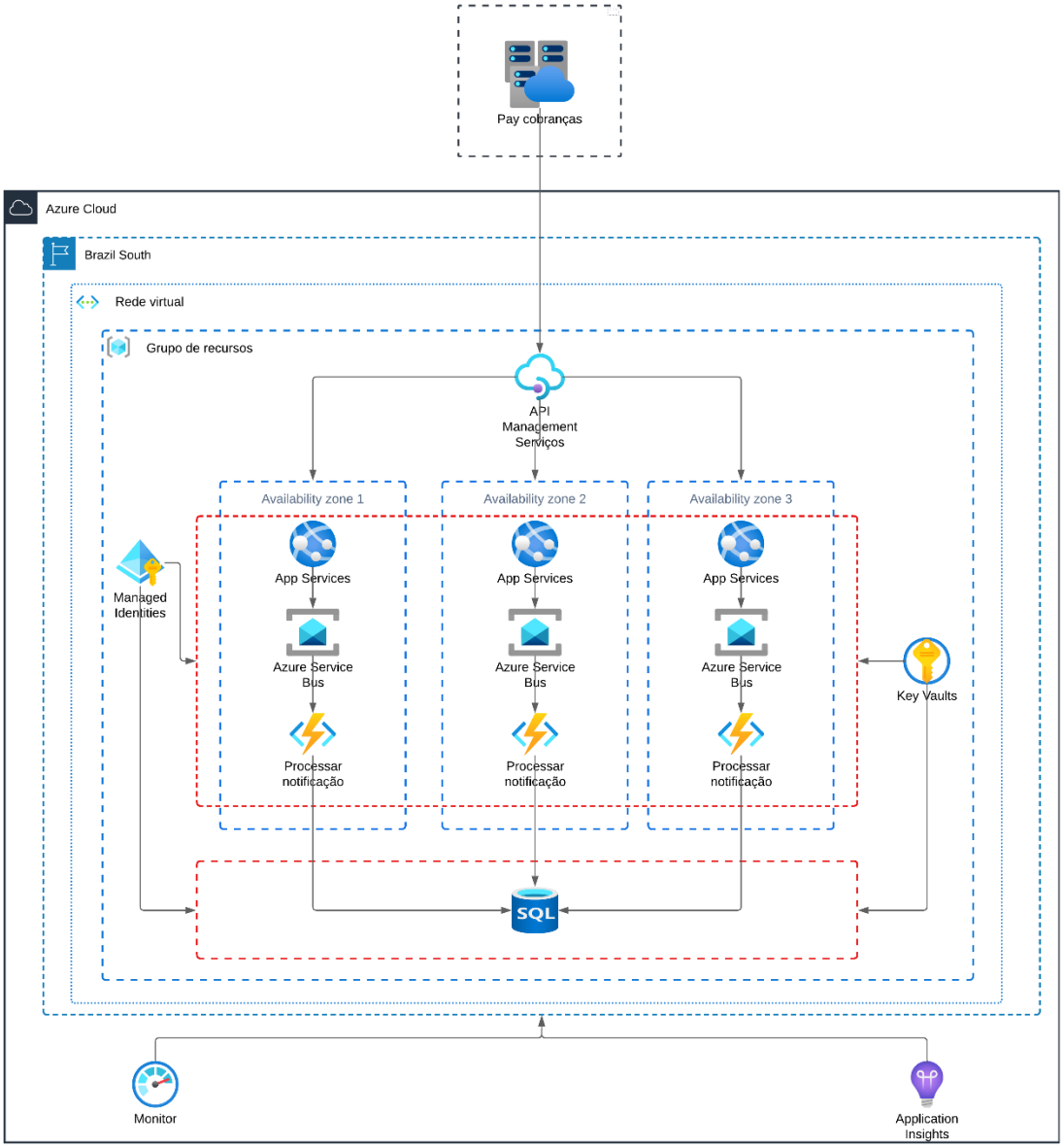
3. Arquitetura da Solução

3.1 Visão Geral da Arquitetura

A solução adota uma arquitetura de microserviços orientada a eventos, com uma API de Webhook na borda, Azure API Management, Azure Service Bus para fila de eventos e funções/processos de backend responsáveis pela conciliação financeira nas bases da Nexus.

- Módulo 1: API
- Módulo 2: Azure services bus
- Módulo 3: Servidor de tarefas

3.2 Diagrama de Arquitetura



4. Componentes Principais

4.1 Backend

API:

- Tecnologia: .NET
- Descrição: Arquitetura desenvolvida utilizando padrão Onion, onde a camada de serviço irá adicionar a fatura na fila do Azure services bus com base no tenant do cliente.

Function:

- Tecnologia: .NET
- Descrição: Arquitetura orientada a evento, onde a mesma irá receber os itens da fila e irá realizar o recebimento nas bases de dados dos clientes.

4.2 Banco de Dados

- Tecnologia: SQL Server
- Modelo de Dados: Modelo Relacional

5. Segurança

5.1 Proteção de Dados

- Criptografia: TLS
- Segregação de dados por tenant em banco, filas e logs; uso de Key Vault para segredos

5.2 Auditoria e Logs

- Logs estruturados por tenant, fatura, tipo de evento e correlação de request/processing

- Métricas: tempo de ponta a ponta do evento, taxa de erro, tamanho da fila, quantidade de mensagens em DLQ, disponibilidade da API

- Alertas configurados para indisponibilidade > 5 minutos, filas acima de limiar, aumento de erros de processamento e quedas de throughput.

6. Escalabilidade e Desempenho

6.1 Escalabilidade Horizontal e Vertical

App Service e Functions com auto-scale configurado por CPU, throughput e profundidade de fila.

Service Bus dimensionado para suportar pelo menos 5x o volume atual de transações, conforme meta de longo prazo.

7. Considerações Finais

7.1 Riscos e Mitigações

- Latência elevada entre regiões: mitigada por escolha de região de baixa latência e testes de desempenho.

- Falhas na integração de dados: mitigadas por uso de filas, DLQ, reprocessamento e auditoria.

- Migração sem foco em segurança: mitigada por TLS, Key Vault, hardening e logs de auditoria.

7.2 Conclusão

- A solução visa implementar microservices de notificação entre a Nexus e Pay cobranças, a arquitetura apresentada aponta uma melhora no processamento, na confiabilidade dos dados e disponibilidade da aplicação.

7.3 Próximos passos:

Detalhar modelos de dados e contratos de mensagens no nível de campo.

Validar volumetria, SLAs e custos em um ambiente de prova de conceito.

Evoluir documentação com diagramas atualizados (C4, topologia) diretamente referenciados neste documento.